

Et kort dokument om L^AT_EX

Hans

Juni 2026

Indhold

1	Hvad er det her?	3
1.1	T _E X, L ^A T _E X og Overleaf	3
1.2	Hvad er det, L ^A T _E X er godt til?	4
1.3	Kom i gang med L ^A T _E X	5
1.3.1	Strukturen af et L ^A T _E X-dokument	5
1.3.2	Arbejdsgangen	6
1.3.3	Filformater	6
1.4	Det der med appendiks	7
2	Om at skrive matematik	8
2.1	Inline vs. display-matematik	8
2.2	Eksponenter, indeks og brøker	8
2.3	Symboler og operatorer	8
2.4	Flerlinjede udregninger	9
2.5	Matricer	9
2.6	Gode råd	9
3	Om at lave figurer	10
4	Algoritmer og kildekode	12
4.1	Algoritmer	12
4.2	Kode	12
5	Om krydsreferencer	14
5.1	Krydsreferencer	14
5.2	Kildehenvisninger	14
6	Gode råd	15
6.1	L ^A T _E X kan meget mere end du tror!	15
6.1.1	Typiske misforståelser	15
6.1.2	Noget om layout	15
A	Her er præamblen	18

Forord

I min tid som censor på de danske datalogiuddannelser har jeg opdaget noget påfaldende: Datalogiuddannelsen på RUC er det eneste sted i den danske universitetsverden, hvor de fleste studerende bruger Microsoft Word i stor stil. De fleste kender ikke til $\text{\LaTeX}$ og ved ikke, hvad dette system tilbyder. Resultatet er kluntet opsatte projektrapporter og underlige, pølsede ad hoc-arbejdsgange, der slet ikke understøtter gruppearbejde godt.

Denne tekst er et forsøg på at skabe en forandring til noget mere passende. Teksten er på en og samme tid

- et eksempel på hvordan man kan bruge $\text{\LaTeX}$ til at skrive dokumenter med, når man studerer datalogi, og
- en guide til hvordan man kommer i gang med at bruge $\text{\LaTeX}$.

Kapitel 1

Hvad er det her?

I afsnit 1.1 forklarer vi forskellen på $\text{\TeX}$, $\text{\LaTeX}$ og Overleaf. Og i afsnit 1.2 forklarer vi, hvorfor man bør bruge disse systemer, når man laver projektarbejde på datalogi. Endelig, i afsnit 1.3, forklarer vi om filformater og hvordan man kommer i gang med at skrive dokumenter i $\text{\LaTeX}$.

1.1 $\text{\TeX}$, $\text{\LaTeX}$ og Overleaf

$\text{\TeX}$ er et dokumentbehandlingssystem, der skyldes den amerikanske datalog og matematiker Donald Knuth. Han brugte $\text{\TeX}$ i sin klassiker, flerbindsværket *The Art of Computer Programming* [Knuth, 1997] og dokumenterede $\text{\TeX}$ i en anden bog [Knuth, 1996]. Knuth fik ACM Turing Award, datalogiens svar på Nobelprisen, for sit arbejde med algoritmeteori, programmeringssprog og meget andet, så han er ikke helt dum.

$\text{\LaTeX}$ er et dokumentbehandlingssystem, der skyldes den amerikanske datalog og matematiker Leslie Lamport. Lamport implementerede $\text{\LaTeX}$ som en samling definitioner af kommandoer og environments, skrevet i $\text{\TeX}$. Han dokumenterede systemet i [Lamport, 1986], der samtidig er en god indføring¹. Knuth fik ACM Turing Award, datalogiens svar på Nobelprisen, for sit arbejde med teorier for specifikation og verifikation af distribuerede systemer, så han er heller ikke helt dum. Nogle mennesker er glade for $\text{\TeX}$, men de fleste bruger $\text{\LaTeX}$.

$\text{\TeX}$ og $\text{\LaTeX}$ kan installeres på din egen computer, og du kan arbejde helt offline med systemet og bruge din yndlings-teksteditor som f.eks. Visual Studio Code eller Emacs sammen med det, præcis ligesom man kan med programmeringssprog. $\text{\TeX}$ Live-distributionen er et godt valg for mange, hvis man vælger denne løsning.

¹Og spørger man mig, er den stadig den bedste af slagsen.

Overleaf er en online-løsning, som blev til i 2011 og skyldes matematikerne John Hammersley og John Lees-Miller. Overleaf er en løsning, der rummer en flerbruger-editor og ikke kræver, at man selv installerer $\text{\TeX}$ / $\text{\LaTeX}$. Mange studerende er glade for Overleaf af disse grunde og kan være et godt sted at starte.

1.2 Hvad er det, $\text{\LaTeX}$ er godt til?

Mange, der ikke har brugt $\text{\LaTeX}$ og lever i Word-land, tror, at $\text{\LaTeX}$ bare er en slags "system til pænt layout med bøvlet brugergrænseflade". Men sandheden er en ganske anden.

Kollaborativ skrivning En gruppe af studerende, der lever i Word-land, skaber ofte en projektrapport, der er en kæmpe stor Word-pølse, som de deler i et Dropbox-katalog eller på anden vis.

Men $\text{\LaTeX}$ understøtter helt direkte modulær skrivning. Man kan dele et stort dokument op i mange små del-filer, som bliver samlet af en main-fil. Dette dokument er et eksempel på dette. $\text{\LaTeX}$ gør det muligt at gøre det, man *egentlig gerne vil* i gruppearbejde. Hvis man i forvejen beskæftiger sig med programmering, ligner dette noget, man kender i forvejen.

Automatisk styr på referencer Noget af det, studerende i Word-land bruger² en masse tid på, er at opdatere krydsreferencer og litteraturhenvisninger i Word-pølsen manuelt. Nogle studerende, der er begyndere udi $\text{\LaTeX}$, tror at de skal være i besiddelse af en stor $\text{\LaTeX}$ -pølse, som de er nødt til at opdatere manuelt. Men $\text{\LaTeX}$ og $\text{\TeX}$ er ud over mulighederne for modularitet designet til at håndtere alt dette automatisk, så alle ændringer kan håndteres automatisk af systemet. Man skal bruge tiden på at tænke over sit projekt, ikke på at skulle håndtere at Figur 17 skal hedde Figur 17 overalt i Word-pølsen.

Simple tekstfiler $\text{\LaTeX}$ -filer er *tekstfiler*, og de er meget små. Selv små Word-pølser fylder temmelig meget. Min PhD-afhandling fylder, alt inklusive, er 124 sider lang og fylder omkring 1 megabyte. Enhver $\text{\LaTeX}$ - eller $\text{\TeX}$ -fil kan åbnes i en vilkårlig teksteditor som Emacs, Vim, Notepad eller Visual Studio Code.

Tekstfiler gør det muligt at bruge versionskontrol som Git til kollaborativ skrivning på præcis samme måde som man kan bruge Git til versionskontrol af programkode.

Bibliografi-databaser $\text{\LaTeX}$ er udviklet sammen med $\text{\BibTeX}$. Alle danske universiteter har universitetsbiblioteker, der tillader at man kan

²eller spilder

hente bibliografi-oplysninger i B_IB_TE_X-format. Også de store bibliografiservere som ACM Digital Library [ACM, 2026] og DBLP [DBLP, 2026] tillader dette. På den måde kan man hente filer med udtømmende oplysninger om sine kilder i stedet for selv at skulle bøvle med at stykke oplysningerne sammen.

Egne kommandoer L_AT_EX er nem at udvide; det er muligt at lave nye kommandoer og environments, så man kan lave nye genveje til de samlinger af kommandoer, man ofte bruger. Hvis man i forvejen beskæftiger sig med programmering, ligner også dette noget, man kender i forvejen.

Pakker Fordi T_EX og L_AT_EX er nemme at udvide, findes der en hel masse udvidelser som gør det nemt at skrive

- programkode (`listings` og `minted`), som vi vender tilbage til i kapitel 4.
- pseudokode (`algorithm`)
- indlejring og skalering af billedfiler (`graphicx`, som vi vender tilbage til senere)
- vektorgrafik (`tikz`, som reelt er et lille sprog i selv!)

At skrive matematik Alle hoved-personerne bag T_EX, L_AT_EX og Overleaf er matematikere og har stor ekspertise inden for matematisk typografi. Skriver man matematik eller inden for andre fagområder, hvor man bruger matematik i stort omfang, er der ikke noget seriøst alternativ. Store sammenslutninger for fagmatematikere som f.eks. American Mathematical Society er med til at udvikle nye pakker til L_AT_EX, og forlag, der udgiver faglitteratur inden for matematik og naturvidenskab (herunder datalogi), kræver som hovedregel, at forfattere bruger L_AT_EX.

1.3 Kom i gang med L_AT_EX

Alle er meget velkomne til at bruge denne tekst som udgangspunkt – det er derfor, den er der.

1.3.1 Strukturen af et L_AT_EX-dokument

En hovedfil består af to

- En *præambel* med pakke-importoplysninger, definitioner af nye kommandoer og environments mm.

- En dokument-del, der er omkranset af `\begin{document}...\end{document}`. I denne del findes selve teksten.

I begge disse dele kan der være `input`-kommandoer, der indlæser underfiler.

1.3.2 Arbejdsgangen

Oprindeligt gjorde implementationer af $\text{T}_{\text{E}}\text{X}$ og $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$ det, at de producerede en fil i `dvi`-format, som så skulle oversættes til en `Postscript`-fil eller en `PDF`-fil.

Nyere implementationer som `pdflatex` springer dette led over og producerer en `PDF`-fil direkte fra `tex`-filen.

Hvis man bruger kommandolinjen Hvis hovedfilen hedder `rapport.tex`, kompilerer man den ved at skrive

```
pdflatex rapport
```

Denne kommando producerer en samling nye filer med forskellige informationer og et (forsøg på) et renderet output. Se Afsnit 1.3.3.

Hvis man bruger Overleaf Hvis man bruger Overleaf, bliver dokumentet altid kompileret tre gange, så citationer og krydsreferencer er blevet samlet rigtigt op.

1.3.3 Filformater

.

tex Filer med dette suffiks indeholder tekst, der bruger $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$ -kommandoer.

aux Filer med dette suffiks indeholder opsamlede data om referencer til figurer, formler, litteratur mm. Disse filer er arbejdsgangens ”hukommelse” og bruges til kompilering.

blg Hvis man bruger $\text{B}_{\text{I}}\text{B}_{\text{T}}\text{E}_{\text{X}}$, er dette en hjælpefil.

bbl Hvis man bruger $\text{B}_{\text{I}}\text{B}_{\text{T}}\text{E}_{\text{X}}$, er dette den bibliografi, der er blevet dannet.

En af filerne er en `aux`-fil, der samler krydsreferencer og kildehenvisninger. Den bliver genereret hver gang man kompilerer. Så hvis man kompilerer for første gang eller ændrer på henvisninger, kan man komme ud for at skulle kompilere én gang til og eventuelt for også at skulle bruge $\text{B}_{\text{I}}\text{B}_{\text{T}}\text{E}_{\text{X}}$ til at samle kildehenvisninger op.

1.4 Det der med appendiks

I en projektrapport eller en artikel har man tit brug for at placere nogle dele af teksten i et appendiks. Sæt kommandoen `\appendix` der, hvor appendikserne skal begynde. Derefter bliver alle kapitlet nummereret med bogstaver.

Kapitel 2

Om at skrive matematik

I L^AT_EX skelner man mellem LR-mode (til brødtekst) og mathmode.

2.1 Inline vs. display-matematik

Matematik skrevet midt i en sætning kalder man *inline*-matematik og skrives mellem dollartegn: $\$ \dots \$$. For eksempel giver $\$f(n) = O(n^2)\$$ resultatet $f(n) = O(n^2)$.

Hvis matematikken skal stå for sig selv på egen linje, bruges *display*-matematik med $\left[\dots \right]$ eller `equation`-omgivelsen (sidstnævnte nummerer udtrykket):

$$T(n) = 2T(n/2) + n \tag{2.1}$$

2.2 Eksponenter, indeks og brøker

Eksponenter skrives med $\wedge$ og indeks med $_$. Bruges der mere end et tegn, skal det sættes i tuborgklammer $\{\}$:

$$x^2, \quad a_i, \quad 2^{n+1}, \quad x_{i,j}$$

Brøker skrives med $\frac{\text{tæller}}{\text{nævner}}$:

$$\frac{n!}{k!(n-k)!}$$

2.3 Symboler og operatorer

`amsmath`-pakken giver adgang til mange nyttige symboler, f.eks.:

$$\sum_{i=1}^n i = \frac{n(n+1)}{2}, \quad \prod_{i=1}^n i, \quad \int_0^1 x^2 dx$$

Mængdesymboler og logik, som ofte bruges i datalogi:

$$\forall x \in \mathbb{N}, \exists y \in \mathbb{Z} : x \leq y, \quad A \cup B, \quad A \cap B, \quad \neg p \vee q$$

2.4 Flerlinjede udregninger

Brug `align`-omgivelsen til at vise flere linjers udregning, linet op efter `&`:

$$(a + b)^2 = a^2 + 2ab + b^2 \quad (2.2)$$

$$= a^2 + b^2 + 2ab \quad (2.3)$$

2.5 Matricer

Matricer skrives med `pmatrix` (parenteser) eller `bmatrix` (kantede parenteser):

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$

2.6 Gode råd

- Indlæs altid pakken `amsmath` – den udvider de grundlæggende matematik-kommandoer betydeligt.
- Brug `\text{...}` til almindelig tekst inde i en matematik-omgivelse, f.eks. n er lige.
- Lad være med at blande `$...$` og `equation` – vælg den, der passer til, om udtrykket skal stå i eller uden for løbende tekst.
- Kompiler ofte, så du hurtigt opdager fejl i din matematik-syntaks.

Kapitel 3

Om at lave figurer

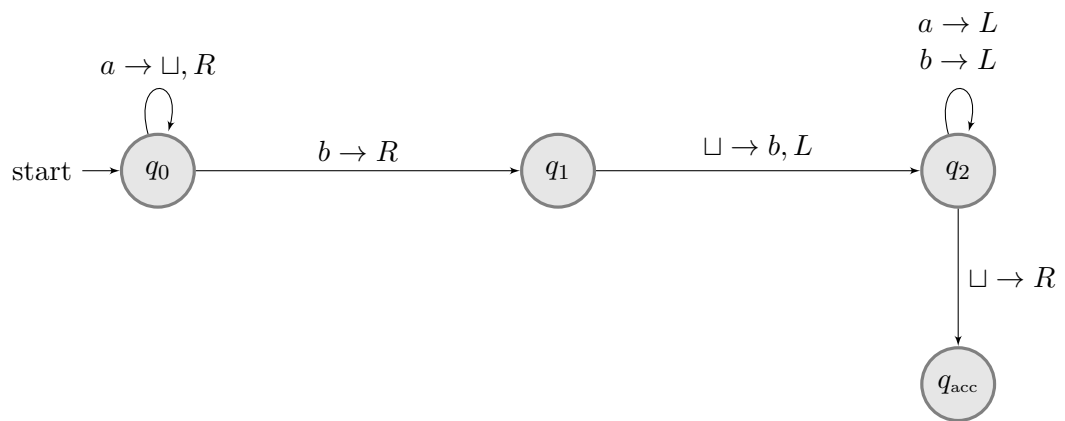
Man kan nemt indlejre grafikfiler med `graphicx`-pakken, der tilbyder bla. `\includegraphics`-kommandoen.

Se Figur 3.1 og læg mærke til, hvordan vi skalerer billedfilen, så den ere 7 centimeter bred i teksten.



Figur 3.1: Leslie Lamport

`tikz` er en pakke, der gør det muligt at beskrive vektorgrafik – det er et helt lille programmeringssprog. Figur 3.2 viser et eksempel, der ligger i en særskilt fil (indlæst med `\input`-kommandoen).



Figur 3.2: Tilstandsdiagram for Turing-maskine

Kapitel 4

Algoritmer og kildekode

4.1 Algoritmer

Man kan skrive algoritmer ved brug af `algorithmicx`-pakken og pseudo-kode ved *også* at bruge `algpseudocode`.

Require: $n \geq 0$

Ensure: $y = x^n$

$y \leftarrow 1$

$X \leftarrow x$

$N \leftarrow n$

while $N \neq 0$ **do**

if N is even **then**

$X \leftarrow X \times X$

$N \leftarrow \frac{N}{2}$

▷ This is a comment

else if N is odd **then**

$y \leftarrow y \times X$

$N \leftarrow N - 1$

end if

end while

4.2 Kode

`listings`-pakken er særdeles nyttig, hvis man skal have kildekode med i sin tekst. Listing 4.1 viser et første lille eksempel på hvad man kan.

```
1 public class Main {
2   public static void main(String[] args) {
3     int time = 16;
4
5     if (time < 12) {
6       System.out.println("Dingo.");
7     } else if (time < 18) {
8       System.out.println("Mango.");
```

```
9      } else {  
10         System.out.println("Bingo.");  
11     }  
12 }  
13 }
```

Listing 4.1: Et eksempel på Java-kode

Man kan også definere sine egne environments, så kildekode i bestemte sprog bliver vist med reserverede ord fremhævet.

Kapitel 5

Om krydsreferencer

Hvis ikke man kender til de rige muligheder for automatisk håndtering af referencer, som $\text{\LaTeX}$ har, er det som at trække rundt på cykelstien med en dyr racercykel og ærgre sig over, at man ikke kan komme hurtigere frem, fordi man er nødt til at trække rundt med hvad man tror er et dårligt designet løbehjul.

5.1 Krydsreferencer

Afsnit, underafsnit, tabeller, figurer og formler kan få numre i $\text{\LaTeX}$. Det gør man med `\label`-kommandoen, som placerer et mærke. Det kan man senere referere til med en `\ref`. Se efter i kildeteksten til dette dokument for at lære, hvordan man laver en henvisning til afsnit 5.2.

Det er aldrig nok at skrive `\ref`, for denne kommando skriver blot nummeret på den label, man henviser til. Jeg har defineret en kommando `\afs`, der tager et mærke som argument og skriver "Afsnit" og nummeret på mærket. Se, hvordan denne kommando er defineret og brugt. Jeg bruger den også her i Afsnit 5.1.

Pakken `cleveref` tilbyder en lignende funktionalitet og er værd at overveje at bruge.

5.2 Kildehenvisninger

Øvelse 5.1. Prøv at skrive

```
\bibliographystyle{apalike}
```

i stedet for `\bibliographystyle{plain}` og se, hvad der sker. Hvis du ikke bruger Overleaf, så husk at kompilere to gange.

Kapitel 6

Gode råd

6.1 L^AT_EX kan meget mere end du tror!

Mange nybegyndere tror, at de skal bruge Word-lignende krumspring for at få et bestemt udseende. Hvis det ikke lykkes for dem, konkluderer de ofte, at L^AT_EX må lide af alvorlige mangler.

6.1.1 Typiske misforståelser

- Måske er det layout, man ønsker sig, ikke hensigtsmæssigt. Det er almindeligt at man har vaner med sig fra gymnasiet og fra Word-land, som snarere skyldes uvaner end stor typografisk indsigt. Det er langt bedre at tage ved lære af typografien i fagbøger og videnskabelige publikationer end at lade sig styre af en gammel SRP-opgave.
- Og der er med meget stor sandsynlighed snarere tale om at der er noget, man ikke ved selv, end om noget, det er umuligt at gøre ved brug af L^AT_EX.

6.1.2 Noget om layout

Hvis man *slet ikke* kan acceptere standard-layoutet og drømmer sig tilbage til SRP-dagene, er der pakken `wordlike`. Her er stor linjeafstand, Times New Roman, smalle marginer og andet godt, så man kan genskabe det trygge gymnasielayout og drømme om Word-pølse.

Øvelse 6.1. Prøv at skrive `\usepackage{wordlike}` i præamblen, og se hvad der sker.

Mange nybegyndere tror også, at de er nødt til at lave manuelle krumspring mange steder i et dokument for at styre dets layout. Typisk ved de ikke, at `report`-dokumentklassen findes og kender kun `article`. Derfor bruger de tvungne sideskift med `\newpage`. Nybegyndere synes også ofte,

at der skal være større afstand mellem paragraffer og indsætter tvungne linjeskift med `\\` overalt.

Dette er en anden typisk arv fra Word-land, hvor man er nødt til at skulle håndtere den slags manuelt rundt om i Word-pølsen. Men i $\text{\LaTeX}$ er der en masse parametre, der bestemmer marginbredde, linjeafstand mm. Dem kan man selv sætte én gang for alle oppe i præamblen, så man ved en lille ændring dér kan ændre dokumentets layout fuldstændig.

Se også https://da.overleaf.com/learn/latex/Articles/How_to_change_paragraph_spacing_in_LaTeX, der forklarer de forskellige parametre, og hvordan man sætter dem ét sted i dokumentet.

Her er nogle små øvelser, der viser hvad parametrene kan.

Øvelse 6.2. Prøv at skrive `\documentclass[a4paper,11pt,twocolumn]{report}` i præamblen og se, hvad der sker.

Øvelse 6.3. Prøv at skrive

```
\setlength{\parskip}{4mm}
```

i præamblen og se, hvad der sker.

Øvelse 6.4. Prøv at skrive

```
\setlength{\parindent}{0mm}
```

i præamblen og se, hvad der sker.

Øvelse 6.5. Prøv at skrive

```
\renewcommand\baselinestretch{1.5}
```

i præamblen og se, hvad der sker.

Litteratur

- [ACM, 2026] ACM (2026). ACM Digital Library. <http://dl.acm.org>.
- [DBLP, 2026] DBLP (2026). DBLP. <http://dblp.org>.
- [Knuth, 1996] Knuth, D. E. (1996). *The T_EXbook*. Computers and Typesetting. Addison-Wesley, Boston, Mass. u. a., incorp. the final corr. edition.
- [Knuth, 1997] Knuth, D. E. (1997). *The Art Of Computer Programming*. Addison-Wesley, Reading, Mass.
- [Lamport, 1986] Lamport, L. (1986). *L^AT_EX: A Document Preparation System*. Addison-Wesley.

Bilag A

Her er præamblen

```
1 \input{pakker}  
2 \input{listings-definitioner}  
3 \input{kommandoer}
```